Malware Detection Framework Using Deep Learning Techniques

Lama Mohammad Aljohani • Seham Lavi Alghamdi • Lujain Emad Hashim • Shahad Mahmoud Alolayan • Reema Akram Al-ahmadi

College of Computer Science and Engineering (Girls Section), Taibah University, Madinah, Saudi Arabia

Supervised by: Dr. Rawan Sanyour | Second Semester, 2026

***Abstract*—The rapid evolution of obfuscated and polymorphic malware has rendered traditional signature-based and heuristic detection methods increasingly ineffective, creating a critical need for intelligent and adaptive cybersecurity solutions. This project presents a deep learning-based malware detection framework that employs image-based analysis of Windows Portable Executable (PE) files. The framework operates through a two-stage pipeline: Stage 1 performs binary classification to distinguish benign files from malicious ones using the MaleX dataset (359,450 Bigram-DCT grayscale images), while Stage 2 conducts multi-class malware family identification across 25 distinct families using the MalImg dataset. Stage 1 uses a CNN augmented with a Squeeze-and-Excitation (SE) attention block, achieving a test AUC of 96.84% and a security-calibrated recall of 94.00% at a threshold of 0.3. Stage 2 uses a CNN Baseline achieving 99.0% validation accuracy and 97.86% test accuracy. The framework is deployed as a Flask REST API with a browser-based interface supporting real-time analysis and downloadable PDF reports.*

***Keywords*—Malware Detection; Deep Learning; Convolutional Neural Network; Bigram-DCT; Squeeze-and-Excitation; Binary Classification; Malware Family Identification; Flask REST API; MaleX; MalImg; Taibah University**

I. INTRODUCTION

Malware is any malicious software intentionally created to damage, disrupt, or gain unauthorized access to computer systems. Common examples include viruses, worms, Trojan horses, ransomware, spyware, and adware. Traditional detection methods — signature-based and heuristic approaches — have become increasingly ineffective against modern and evolving malware, as attackers use advanced obfuscation and code transformation techniques to bypass conventional antivirus systems [1], [2]. This creates a strong need for more intelligent and adaptive detection mechanisms.

This project introduces an artificial intelligence (AI) based malware detection system that combines image processing with deep learning. By converting executable files into Bigram-DCT grayscale images and training Convolutional Neural Network (CNN) models, the system automatically recognizes hidden patterns and classifies files as benign or malicious — and further identifies the specific malware family for deeper threat intelligence.

The main contributions of this work are:

- A two-stage deep learning pipeline that combines binary malware detection (Stage1) with fine-grained malware family classification into 25 families (Stage 2) in a unified end-to-end system.
- Integration of a Squeeze-and-Excitation (SE) attention block into the Stage 1 CNN for channel-wise feature recalibration, selectively amplifying discriminative frequency components.
- A systematic threshold analysis showing that a security-oriented threshold of 0.3 raises malware recall from 89.92% to 94.00%, reducing undetected threats at an acceptable precision cost.
- A Flask REST API deployment with ngrok HTTPS tunnel, browser-based UI supporting drag-and-drop upload, real-time classification, and downloadable PDF report export.

II. RELATED WORK

A. Classical Machine Learning Approaches

Early malware detection relied on handcrafted features derived from static analysis. N-gram-based methods [6] extract byte or opcode sequences and feed them into classifiers such as SVM or Random Forest. Kancherla and Mukkamala [7] introduced binary-to-image visualization using intensity, texture, Gabor, and Wavelet features with SVM, achieving 95% accuracy on ~37,000 samples. While foundational, these approaches require extensive feature engineering and fail to generalize to obfuscated variants.

B. Deep Learning for Malware Detection

Kumar et al. [3] demonstrated that a 3-layer CNN on 128×128 grayscale malware images achieves 98% accuracy on MalImg without manual feature engineering. Palomino [8] showed that Convolutional Recurrent Neural Networks (CRNNs) improve over standard CNNs (67.8% → 92.24%) by capturing temporal dependencies. El Abdelkhalki et al. [9] showed ResNet-152 with ImageNet transfer learning achieves 93.50% accuracy and AUC 0.94 at higher computational cost. Rajuroy [4] reported 95% accuracy on MalImg with CNNs and F1=0.92 on EMBER with hybrid CNN+RNN models.

More recently, Vision Transformer (ViT) architectures have been applied. Belal et al. [10] achieved up to 98.92% accuracy with multi-variant ViT and MCDM model selection. Alsubaie et al. [11] proposed LeViT-MC achieving 96.6% accuracy at 2,370 images/second. Despite these advances, prior works predominantly validate on single datasets and assume fully supervised learning, limiting cross-dataset generalizability.

III. METHODOLOGY

A. Bigram-DCT Image Representation

Following Mohammed et al. [5], executable binaries are converted to 256×256 Bigram-DCT grayscale images. Each binary is read as a hexadecimal byte stream; all overlapping bigrams (2-byte pairs) are extracted and a 256×256 frequency count matrix is built where entry (i,j) holds the normalized count of bigram (i,j). The 0000 bigram count is zeroed out as it represents padding with no semantic value. A full-frame Discrete Cosine Transform (DCT) is applied to de-sparsify the matrix, yielding a Bigram-DCT image with distinctive texture patterns reflecting the internal byte-level structure of the binary. This frequency-domain representation is robust to local byte permutations common in obfuscated malware and produces a fixed-size 256×256 image regardless of the original file size.

B. Stage 1 — Binary Detection

Stage 1 uses the MaleX dataset [5]. A balanced subset of 359,450 Bigram-DCT images was constructed: 179,725 benign + 179,725 malware. The dataset was partitioned using stratified splits: 70% train (251,615), 15% validation (53,917), 15% test (53,918). Zero data leakage was confirmed across all splits.

The Stage 1 model is a CNN enhanced with an SE attention block. Four convolutional blocks (32→64→128→256 filters) with Batch Normalization and Max Pooling are followed by an SE block that squeezes global spatial information via Global Average Pooling, learns channel importance through two Dense layers (ReLU + sigmoid), and rescales the input features accordingly. The classification head uses two Dense layers (256 and 128 neurons) with Dropout (0.4 and 0.3), followed by a sigmoid output. Total parameters: 497,041. Compiled with Adam (lr=1e-3), Binary Cross-Entropy, batch size 64, trained up to 40 epochs with ModelCheckpoint (val_auc), ReduceLROnPlateau (factor=0.5, patience=3), and EarlyStopping (patience=5) on an NVIDIA A100 GPU.

C. Stage 2 — Malware Family Classification

Stage 2 uses MalImg [12]: 9,339 grayscale PNG images across 25 malware families, pre-partitioned into 6,537 train / 1,401 validation / 1,401 test. Images were resized to 224×224, converted from single-channel grayscale to 3-channel via channel replication, and pixel-normalized to [0, 1]. Data augmentation (random flips, brightness, contrast) was applied to training only.

Three CNN architectures were evaluated: (1) CNN Baseline — four-block CNN (32→64→128→256), GAP, dense head 512→256, Dropout 0.5/0.3, 661,721 parameters; (2) CNN+SE Block — adds SE attention after each block, 670,185 parameters (+1.3%); (3) MobileNetV2 Transfer Learning — pre-trained ImageNet backbone, two-phase: frozen 15 epochs (lr=1e-3), fine-tune last 30 layers 25 epochs (lr=1e-5), 3,053,657 parameters.

D. Threshold Selection and Security Rationale

In security-critical deployments, a False Negative — undetected malware — is far more dangerous than a False Positive. An undetected malware sample can result in system compromise, data exfiltration, or ransomware execution; a false alarm results only in additional inspection. Following systematic evaluation at thresholds 0.3, 0.4, and 0.5 on the Stage 1 test set, a threshold of 0.3 was adopted, raising recall from 89.92% to 94.00% at a precision of 86.38%. This decision was agreed with the project supervisor.

E. Integrated Pipeline and Deployment

The two models are integrated into pipeline_predict() with an early-exit design: Stage 1 runs first at threshold 0.3; benign files terminate immediately (no Stage 2 invoked); malware files are forwarded to Stage 2. If Stage 2 family confidence falls below 0.5, the result is returned as 'Unknown Malware — Family not recognized', preventing overconfident misclassification. The system was deployed as a Flask REST API (port 5001, ngrok HTTPS tunnel), with permitted extensions {.exe, .bin, .dll, .sys, .dat, .png}, automatic binary-to-image conversion at inference, and temporary file cleanup. A browser-based HTML/CSS/JavaScript frontend communicates via asynchronous Fetch API requests.

IV. EXPERIMENTS AND RESULTS

A. Stage 1 — Binary Detection Results

Table I reports Stage 1 test performance. The CNN+SE model achieved test AUC 96.84% and accuracy 91.00%. The confusion matrix shows 24,695 TN, 24,242 TP, 2,264 FP, 2,717 FN at default threshold 0.5. The ROC curve rises steeply from the origin, demonstrating high TPR at very low FPR — well above the random baseline (AUC=0.5).

TABLE I: Stage 1 Test Set Performance (Threshold 0.5)

Metric	Value
Test Accuracy	91.00%
Test AUC	96.84%
Precision	91.46%
Recall	89.92%
F1-Score	**90.68%**

Table II presents threshold comparison. At threshold 0.3, recall increases to 94.00% — the operational configuration selected for deployment.

TABLE II: Threshold Comparison on Stage 1 Test Set

Threshold	Precision	Recall	F1-Score
0.3 (Selected)	86.38%	94.00%	90.03%
0.4	89.19%	92.00%	90.57%
0.5 (Default)	**91.46%**	**89.92%**	**90.68%**

B. Stage 2 — Family Classification Results

Table III compares the three Stage 2 architectures. The CNN Baseline achieved the highest validation accuracy of 99.0% and test accuracy of 97.86%, outperforming both CNN+SE and MobileNetV2. This confirms that for the visually well-structured MalImg dataset, a well-regularized four-block CNN provides an optimal balance between feature extraction and generalization. Increased model complexity did not improve performance.

TABLE III: Stage 2 Model Comparison

Model	Best Val Acc	Test Acc	Selected
CNN Baseline	99.00%	97.86%	✓
CNN + SE Block	98.93%	98.57%	—
MobileNetV2 Transfer	**98.57%**	**97.50%**	**—**

Table IV shows the per-family classification report for the selected CNN Baseline. Near-perfect F1-scores (1.00) were achieved for 18 of 25 families. Weighted average precision and recall are 0.97 and 0.98 respectively. The two lowest-performing families — Swizzor.gen!E (F1=0.44) and Swizzor.gen!I (F1=0.35) — share a common codebase and produce nearly indistinguishable byte-level texture patterns in grayscale, representing a fundamental limitation of purely static image-based classification.

TABLE IV: Stage 2 Classification Report (Selected Families, CNN Baseline)

Family	Precision	Recall	F1	Support
Adialer.C	1.00	1.00	1.00	18
Agent.FYI	1.00	0.89	0.94	18
Allaple.A	1.00	1.00	1.00	443
Allaple.L	1.00	1.00	1.00	239
Alueron.gen!J	1.00	1.00	1.00	29
C2LOP.P	0.82	0.82	0.82	22
C2LOP.gen!g	0.88	0.97	0.92	30
Swizzor.gen!E	0.41	0.47	0.44	19
Swizzor.gen!I	0.43	0.30	0.35	20
VB.AT	1.00	1.00	1.00	61
Yuner.A	1.00	1.00	1.00	120
Weighted Avg	**0.97**	**0.98**	**0.97**	**1,401**

C. System Testing

The deployed Flask API was tested via Postman with five test cases covering: malware detection with family identification, benign file filtering, and unsupported file rejection. Three malware samples (Agent.FYI, Dontovo.A, Lolyda.AA2) were correctly classified at Stage 1 with scores of 1.0, then identified by Stage 2 with confidence 0.9982, 1.0, and 0.7009 respectively. One benign file was correctly filtered at Stage 1 (score: 0.0245, Stage 2 not invoked). An unsupported format was rejected with HTTP 400. Average response time: 2.3 seconds.

TABLE V: System Test Results Summary

Test Case	Stage 1 Result	Stage 2 Result	HTTP	Time
Agent.FYI	Malware (1.0)	Agent.FYI (0.998)	200	3.31s
Dontovo.A	Malware (1.0)	Dontovo.A (1.0)	200	2.13s
Lolyda.AA2	Malware (1.0)	Lolyda.AA2 (0.70)	200	1.44s
Benign file	Benign (0.02)	Not invoked	200	0.91s
Unsupported	—	—	**400**	—

V. DISCUSSION

The initial Stage 1 implementation operated directly on MaleX's pre-extracted 320-dimensional numerical feature vectors, yielding only ~73% validation accuracy. This limitation was identified as a fundamental incompatibility between flat vectors and CNN spatial feature extraction. Redesigning Stage 1 to reconstruct Bigram-DCT images from source files resolved this, improving test accuracy to 91.00% and AUC to 96.84%.

The SE attention block delivered channel-wise feature recalibration with minimal overhead (<1.3% additional parameters), replacing the originally proposed Multiple Instance Learning (MIL) component. MIL was inapplicable because MaleX provides fixed-size, fully labeled images rather than unlabeled bags of patches. Similarly, Adaptive Learning Rate (ALR) was replaced by ReduceLROnPlateau, which proved more effective for the low-variance gradient distributions characteristic of DCT malware imagery.

For Stage 2, the CNN Baseline outperformed both CNN+SE (+1.3% params) and MobileNetV2 (+363% params). A fused MalImg+MalNet dataset was initially attempted, resulting in 721 classes with average 77 samples/class and best validation accuracy of only 57.64% — confirming that MalImg alone provides sufficient class balance and visual structure for reliable CNN training. The test accuracy of 97.86% is comparable to 97.18% reported by Nataraj et al. [12] with GIST+k-NN and 98.00% by Kumar et al. [3] with a 3-layer CNN on the same dataset.

VI. LIMITATIONS AND FUTURE WORK

Three principal limitations constrain the current implementation. First, binary-to-image conversion at inference introduces structural fragility: any inconsistency in DCT normalization or matrix sizing between training and deployment degrades Stage 1 predictions. Future work should explore format-agnostic representations such as raw byte sequences processed by 1D CNNs or transformer architectures.

Second, Swizzor.gen!E and Swizzor.gen!I achieved substantially lower F1-scores (0.44 and 0.35) due to their shared codebase and nearly indistinguishable grayscale textures. Hybrid approaches integrating static image analysis with behavioral features (API call sequences, memory access patterns, network behavior) should be explored to improve discrimination between structurally similar variants.

Third, deployment within Google Colab with a dynamic ngrok tunnel is unsuitable for continuous production use. Migration to a containerized cloud infrastructure (Docker + Kubernetes) with a persistent HTTPS endpoint would enable continuous availability and horizontal scalability.

VII. CONCLUSION

This paper presented a deep learning-based malware detection framework that integrates Bigram-DCT image visualization with a two-stage CNN pipeline for binary detection and malware family classification. Stage 1 achieved a test AUC of 96.84% and a security-calibrated recall of 94.00% at threshold 0.3 on a balanced MaleX subset of 359,450 images. Stage 2 achieved 97.86% test accuracy across 25 malware families using the MalImg dataset, with the CNN Baseline outperforming both SE-augmented and transfer learning architectures. The framework was successfully deployed as a Flask REST API with a browser-based user interface. These results demonstrate that a well-engineered, moderately sized CNN — combined with appropriate preprocessing, threshold calibration, and deployment infrastructure — can achieve detection performance that is both practically reliable and academically competitive with the state-of-the-art.

ACKNOWLEDGMENT

The authors express sincere gratitude to Dr. Rawan Sanyour for her unwavering guidance and deep expertise throughout this research. The authors also acknowledge the researchers who made the MalImg and MaleX datasets publicly available, and the Information Systems Department at Taibah University for providing the academic environment and technical support that made this project possible.

REFERENCES

[1] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: visualization and automatic classification," in Proc. 8th Int. Symp. Visualization for Cyber Security (VizSec), ACM, 2011, pp. 4:1–4:7.

[2] L. Nataraj and B. Manjunath, "SPAM: Signal processing to analyze malware," IEEE Signal Processing Magazine, vol. 33, no. 2, pp. 105–117, 2016.

[3] A. Kumar et al., "CNN-based malware image classification," in Proc. IEEE ICMLA, 2018.

[4] Rajuroy, "CNN and hybrid models for malware detection on Malimg and EMBER datasets," 2022.

[5] T. M. Mohammed, L. Nataraj, S. Chikkagoudar, S. Chandrasekaran, and B. S. Manjunath, "Malware Detection Using Frequency Domain-Based Image Visualization and Deep Learning," arXiv:2101.10578, 2021.

[6] J. Jang, D. Brumley, and S. Venkataraman, "Bitshred: Feature hashing malware for scalable triage," in Proc. 18th ACM CCS, 2011, pp. 309–320.

[7] K. Kancherla and S. Mukkamala, "Image visualization based malware detection," in Proc. IEEE SSCI, 2013.

[8] J. Palomino, "Deep learning architectures for image-based malware detection," arXiv, 2023.

[9] I. El Abdelkhalki et al., "Deep ResNet with transfer learning for malware classification," 2022.

[10] M. Belal et al., "Multi-variant ViT with MCDM for malware detection," IEEE Access, 2023.

[11] A. Alsubaie et al., "LeViT-MC: Efficient transformer for malware classification," 2023.

[12] L. Nataraj, S. Karthikeyan, G. Jacob, and B. Manjunath, "Malware images: visualization and automatic classification," in Proc. 8th Int. Symp. VizSec, ACM, 2011.